

SRS ref

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name

Context Assembly Framework (CAF)

Classification

Internal Engineering Tool – Confidential
For Internship Development Use Only

Technology Stack

- Backend: Python
- UI Layer: Streamlit
- Storage: JSON / SQLite (local)

1. INTRODUCTION

1.1 Purpose

The Context Assembly Framework (CAF) is an **internal instruction-structuring system** designed to standardize how complex instructions are captured, organized, versioned, and reused across multiple downstream systems.

The framework focuses on:

- Structured input collection
- Deterministic instruction assembly
- Versioning and iteration tracking
- Feedback signal capture

CAF is **not a decision-making system** and does **not perform optimization or learning** in the current phase.

1.2 Design Intent (Important)

CAF is designed as a **generic infrastructure layer**, independent of:

- Business domain
- Industry use case
- Specific AI models
- External APIs

All intelligence, optimization, and decision logic is handled **outside** this framework.

2. SCOPE

2.1 In Scope

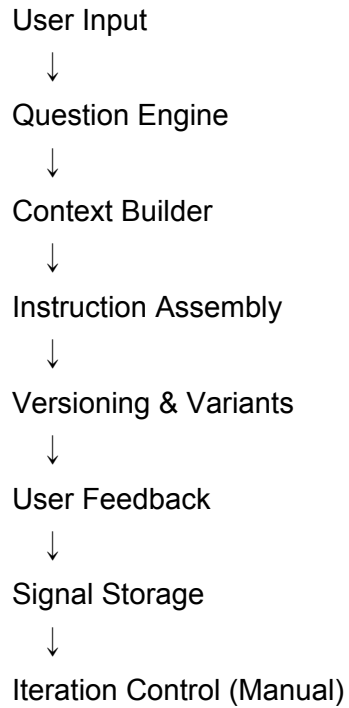
- Structured question-driven input capture
- Context normalization and validation
- Instruction assembly using templates
- Version control for assembled instructions
- Iteration tracking and history
- Feedback and signal logging
- Controlled regeneration workflows

2.2 Out of Scope (Explicit)

- Automatic optimization or self-modification
- Domain-specific rules or logic
- Machine learning or reinforcement learning models
- External AI / LLM API integrations
- Business or product-specific configurations

3. SYSTEM OVERVIEW

3.1 High-Level Workflow



4. FUNCTIONAL REQUIREMENTS

4.1 Question Intelligence Engine

Objective

Collect structured information required to assemble clear and consistent instructions.

Functional Requirements

- Questions must be grouped into categories:
 - Task Objective
 - Target Audience
 - Tone & Style
 - Constraints & Limitations
 - Output Structure
 - Risk / Sensitivity Level
- Support the following:
 - Mandatory and optional questions
 - Conditional questions (based on prior answers)
 - Multiple input types (text, select, multi-select)

Question Definition Format (JSON)

```
{
  "question_id": "task_objective",
  "label": "Describe the primary task objective",
  "type": "text",
  "required": true,
  "category": "objective"
}
```

4.2 Context Builder Module

Objective

Convert raw answers into a **normalized context object**.

Context Object Schema

```
{
  "objective": "",
  "audience": "",
  "tone": "",
  "constraints": [],
  "output_format": "",
  "risk_level": "low | medium | high"
}
```

Requirements

- Context must be:
 - Validated
 - Editable
 - Stored independently
- No inference or interpretation is allowed
- Values must reflect user input only

4.3 Instruction Assembly Engine

Objective

Assemble structured instructions using predefined templates.

Requirements

- Template-based assembly only
- Deterministic output (no randomness)
- Full transparency of assembled instruction
- Users must be able to manually edit final output

Example Template

Instruction Objective:

{{objective}}

Target Audience:

{{audience}}

Constraints:

{{constraints}}

Tone:

{{tone}}

Expected Output Format:

{{output_format}}

4.4 Version Control System

Objective

Maintain complete history of assembled instructions.

Requirements

- Each instruction must have:
 - Version number
 - Creation timestamp
 - Change description
- Support:
 - Version comparison (diff)
 - Rollback to previous versions
 - Manual edits tracked per version

4.5 Variant Generation Module

Objective

Generate multiple structural variants of the same instruction.

Requirements

- Variants must:
 - Share the same base context
 - Differ only in structure or ordering
- Variants must be linked to a parent instruction
- No automatic ranking or selection of variants

4.6 Feedback & Signal Capture

Objective

Capture user feedback and iteration signals for analysis.

Signals to Capture

- User rating (1–5)
- Feedback tags (e.g., unclear, verbose, incomplete)
- Whether manual edits were made
- Number of regeneration cycles
- Time taken to finalize

Signal Storage Format

```
{  
  "instruction_id": "",  
  "version": "",  
  "rating": 3,  
  "feedback_tags": ["verbose"],  
  "manual_edits": true  
}
```

4.7 Iteration & Loop Control

Objective

Enable controlled re-iteration without autonomous behavior.

Phase-1 Behavior

- Users can:
 - Modify context
 - Regenerate instruction
 - Create new variants
- System maintains:
 - Iteration chain
 - Parent-child relationships
 - Full audit trail

Note

The system **must not** make decisions or apply feedback automatically.

5. DATA STORAGE

Phase-1 Storage

- SQLite or JSON-based local storage
- Entities:
 - Questions
 - Context objects
 - Instructions
 - Versions
 - Variants
 - Feedback signals
 - Iteration links

6. NON-FUNCTIONAL REQUIREMENTS

Performance

- Instruction assembly < 1 second
- Smooth UI interaction

Security

- No external connectivity
- No real business data
- Internal deployment only

Maintainability

- Modular Python code
- Clear separation of:
 - UI
 - Logic
 - Storage
 - Configuration

7. ACCEPTANCE CRITERIA

The system is considered complete if:

- Structured inputs can be collected via questions
- Context is generated and editable
- Instructions are assembled deterministically
- Versions and variants are tracked
- Feedback is stored consistently
- No autonomous decision-making exists

8. FUTURE READINESS (DESIGN NOTE)

CAF is designed to later support:

- Rule-based optimization
- Human-approved reinforcement
- Comparative analysis of variants
- Model-assisted suggestions

These features are **explicitly excluded from Phase 1**.

9. OWNERSHIP & CONFIDENTIALITY

All code, documentation, and outputs produced under this project are the exclusive intellectual property of the Company and are governed by signed internship IP and confidentiality agreements.

INTERNAL NOTE (DO NOT DISCUSS OUTSIDE)

This system is an **infrastructure layer**, not a standalone product.
Usage, intelligence, and optimization logic are managed separately.

Phase Wise Division

Context Assembly Framework (CAF)

Phase-Wise Execution Plan

Phase 1: Conceptual Orientation and System Boundary Understanding

In this phase, the team is expected to develop a clear understanding of what the Context Assembly Framework (CAF) is intended to be and, equally importantly, what it is not. The focus is on understanding CAF as a deterministic instruction-structuring framework rather than an intelligent, adaptive, or decision-making system. The team should study the system description and explicitly internalize that CAF does not perform inference, optimization, ranking, learning, or autonomous behavior of any kind. The objective of this phase is to establish strong conceptual boundaries so that all future implementation decisions remain strictly aligned with the defined scope. By the end of this phase, the team should be able to explain CAF's purpose, scope, and limitations clearly and consistently.

Phase 2: Reference Tool Exploration for Structural Awareness

This phase focuses on exposure to external tools strictly for observational and design awareness purposes. The team explores tools such as Google Anti-Gravity and Google Stitch to understand how structured input collection, guided workflows, and modular UI components are presented to users. The emphasis is on observing clarity of flow, organization of inputs, and configuration-driven behavior. The team must not attempt to replicate functionality or internal logic from these tools. Instead, this phase is limited to extracting high-level structural and UI organization ideas that can inform CAF's presentation layer without introducing inference, automation, or decision logic.

Phase 3: Technology Stack Familiarization

In this phase, the team becomes familiar with the selected technology stack used in CAF. This includes understanding Python project structuring with a strong emphasis on modularity and

separation of concerns. Streamlit is explored as a UI layer for building structured, form-based user interactions. The team also studies local persistence mechanisms using JSON files and SQLite databases. The intent of this phase is to ensure that the team understands how data flows cleanly from UI components to logic layers and into storage, without embedding business rules or intelligence at any layer.

Phase 4: System Decomposition and Module Identification

This phase involves decomposing CAF into clearly defined internal modules. The team identifies and documents each major component, including the Question Engine, Context Builder, Instruction Assembly Engine, Version Control System, Variant Management, Feedback Capture, and Iteration Control. For each module, the team defines its responsibilities, inputs, outputs, and explicit boundaries. This decomposition ensures a modular system design and prevents unintended coupling or scope creep during implementation.

Phase 5: Question Engine Design and Definition

During this phase, the team designs the Question Engine responsible for collecting structured information from users. Questions are organized into predefined categories such as task objective, target audience, tone and style, constraints and limitations, output structure, and risk level. Each question is defined using a JSON-based schema that specifies identifiers, labels, input types, required flags, and conditional visibility rules. The focus is on making the question system declarative and configurable, ensuring that changes can be made through configuration rather than code changes, and without embedding interpretation logic.

Phase 6: Dynamic Question Rendering and Input Capture

In this phase, the team implements logic to dynamically render questions in the user interface based on their definitions. Mandatory questions must be enforced, optional questions handled correctly, and conditional questions displayed only when their defined conditions are met. The team ensures that all user inputs are captured accurately and consistently. Progression through the flow must be blocked if required inputs are missing or invalid. At this stage, the system functions purely as a structured data collection interface.

Phase 7: Answer Validation and Editing Support

This phase focuses on validating collected answers and providing full transparency and control to users. The team implements validation rules to ensure that required fields are completed and that values conform to expected formats or allowed ranges. Users must be able to review all provided answers and make edits before finalizing them. This phase emphasizes correctness, user control, and explicit confirmation of all captured data.

Phase 8: Context Builder Construction

In this phase, the team implements the Context Builder, which converts validated user answers into a normalized context object based on a predefined schema. The Context Builder performs no inference, enrichment, transformation, or interpretation of data. All values in the context object must directly reflect user-provided inputs. The system must also allow users to manually edit context fields after construction. Each context object is stored independently with a unique identifier to support traceability and auditability.

Phase 9: Instruction Template Definition

During this phase, the team defines clear, readable instruction templates that contain placeholders corresponding to context fields. These templates represent the structural blueprint for assembling instructions. The emphasis is on transparency, ensuring that users can clearly see how context values are mapped into the final instruction output. Templates must remain static and human-readable.

Phase 10: Instruction Assembly Engine Implementation

In this phase, the team implements deterministic logic to assemble instructions by substituting context values into predefined templates. The output must be fully deterministic, meaning the same input always produces the same output. Users are given the ability to manually edit the assembled instruction. Any manual edits are explicitly tracked and recorded. The system must not modify, enhance, or reinterpret instruction content automatically.

Phase 11: Version Control and History Management

This phase introduces version control for assembled instructions. The team implements mechanisms to create a new version whenever an instruction is regenerated or manually edited. Each version includes metadata such as version number, creation timestamp, and a user-provided change description. The system also supports comparison between versions, allowing users to view differences and roll back to previous versions if required. All versions remain immutable once created.

Phase 12: Variant Creation and Management

In this phase, the team enables users to create multiple variants of an instruction derived from the same base context. Variants differ only in structure or ordering of sections, not in semantic content. Each variant is linked to its parent instruction and associated context. The system does not perform any ranking, evaluation, or selection of variants; all variants are treated as equal alternatives and are user-managed.

Phase 13: Feedback and Signal Capture

This phase focuses on capturing user feedback and interaction signals. The team implements mechanisms to record ratings, predefined feedback tags, and indicators of whether manual edits were made. Feedback data is stored independently and linked to specific instruction versions. The system does not react to feedback or alter behavior based on it; feedback is captured strictly for record-keeping and analysis purposes.

Phase 14: Iteration and Regeneration Control

In this phase, the team implements controlled iteration workflows. Users can modify context values, regenerate instructions, or create new variants. The system tracks iteration chains and maintains clear parent-child relationships between instructions, versions, and variants. All actions are explicitly user-initiated. No automatic looping, branching, or decision-making behavior is permitted.

Phase 15: Data Storage Finalization and Integrity Enforcement

During this phase, the team finalizes storage structures for all system entities, including questions, contexts, instructions, versions, variants, feedback records, and iteration links. Referential integrity is enforced to ensure that all records remain consistent and traceable. This phase ensures that the system is auditable, reliable, and suitable for internal use.

Phase 16: Internal Validation and Completion Review

In the final phase, the team validates the system against the acceptance criteria defined in the SRS. This includes confirming that structured inputs can be collected, contexts are editable, instructions are assembled deterministically, versions and variants are tracked correctly, feedback is stored consistently, and no autonomous behavior exists anywhere in the system. Internal documentation is prepared to describe system structure, data schemas, and usage guidelines.

Lead-Wise Execution Responsibility

Lead-Wise Execution Responsibility

Context Assembly Framework (CAF)

Lead-Wise Execution Responsibility

Lead 1: System Foundations & Boundary Compliance Lead

This lead is responsible for ensuring that the team fully understands the purpose, scope, and constraints of the Context Assembly Framework before any implementation begins. The focus of this responsibility is to establish and enforce clear system boundaries throughout development.

This lead ensures that the team internalizes CAF as a deterministic instruction-structuring framework and not as an intelligent, adaptive, or decision-making system. They are responsible for validating that all implementation decisions align strictly with the documented scope and that no features introduce inference, automation, ranking, learning, or autonomous behavior.

This lead also oversees system decomposition, ensuring that all modules are clearly defined with well-understood responsibilities and boundaries. Any ambiguity regarding what a module can or cannot do must be resolved under this lead's supervision.

Owns:

- Conceptual orientation
- Scope enforcement
- System boundary validation
- Module responsibility clarity

Lead 2: UI & Interaction Flow Lead (Streamlit)

This lead owns the user interaction layer of CAF. Their responsibility is to ensure that structured input collection is implemented clearly, consistently, and transparently using Streamlit.

This includes dynamic rendering of questions, enforcement of mandatory and optional inputs, conditional question visibility, and smooth navigation across the input flow. The lead ensures that the UI does not embed business logic, interpretation, or automation, and that it acts purely as a presentation and input capture layer.

They are also responsible for maintaining a clean separation between UI components and backend logic, ensuring that all UI behavior is driven by configuration and data definitions rather than hardcoded rules.

Owns:

- Streamlit UI architecture
- Dynamic question rendering
- Input capture workflows
- UI-state management
- UI-logic separation

Lead 3: Question Engine & Schema Lead

This lead is responsible for the design and implementation of the Question Engine. Their focus is on defining how questions are structured, categorized, stored, and interpreted strictly as data.

They oversee the creation of the question JSON schema, ensuring that each question includes identifiers, labels, input types, required flags, categories, and conditional rules. The lead ensures that the question system remains declarative and configurable, allowing changes without code modification.

They also ensure that no interpretation or inference is embedded into the question definitions or evaluation logic.

Owns:

- Question taxonomy
- Question JSON schema
- Conditional question rules
- Declarative configuration design

Lead 4: Validation & Context Builder Lead

This lead owns the transformation of user answers into structured, normalized context objects. Their responsibility is to ensure that validation rules are correctly applied and that only explicitly provided user inputs are included in the context.

They oversee the validation layer that enforces required fields, acceptable values, and data consistency. They also ensure that users can manually edit context fields after construction and that all changes are explicitly user-driven.

This lead guarantees that the Context Builder performs no inference, enrichment, or transformation beyond structural normalization.

Owns:

- Input validation logic
- Context object schema
- Context construction
- Manual context editing support
- Context persistence

Lead 5: Instruction Assembly & Template Lead

This lead is responsible for instruction template definition and deterministic instruction assembly. They ensure that templates are clear, readable, and transparent, with placeholders that directly map to context fields.

They oversee the instruction assembly engine, ensuring that it produces fully deterministic outputs. This lead also ensures that manual edits to assembled instructions are supported and explicitly tracked, without any automatic modification or enhancement by the system.

Owns:

- Instruction template design
- Deterministic assembly logic
- Manual instruction editing
- Edit tracking mechanisms

Lead 6: Version Control & History Lead

This lead owns the versioning system for instructions. Their responsibility is to ensure that every regeneration or manual edit results in a new immutable version with proper metadata.

They oversee version creation workflows, metadata management, version comparison (diff), and rollback mechanisms. This lead ensures that version history is complete, auditable, and transparent.

Owns:

- Version metadata schema
- Version creation logic
- Version comparison views
- Rollback functionality
- Immutable history enforcement

Lead 7: Variant Management Lead

This lead is responsible for implementing and managing instruction variants. Their role is to ensure that variants are created correctly as structural or ordering alternatives derived from the same base context.

They oversee variant linkage to parent instructions and contexts and ensure that variants are stored with clear lineage. This lead enforces the rule that the system does not rank, evaluate, or select variants in any way.

Owns:

- Variant definition rules
- Parent–child linking
- Variant storage and retrieval
- Lineage tracking

Lead 8: Feedback & Signal Capture Lead

This lead owns the feedback and signal capture mechanisms. Their responsibility is to ensure that user ratings, feedback tags, manual edit indicators, and related signals are captured accurately and stored consistently.

They ensure that feedback data is linked to the correct instruction versions and remains isolated from system behavior. No system response or adaptation based on feedback is permitted.

Owns:

- Feedback schema

- Signal capture workflows
- Feedback persistence
- Audit traceability

Lead 9: Iteration & Audit Trail Lead

This lead is responsible for managing controlled iteration workflows. They ensure that users can modify context, regenerate instructions, and create variants while maintaining a complete and traceable iteration chain.

They oversee parent-child relationship tracking across contexts, instructions, versions, and variants. This lead ensures that all actions are user-initiated and fully auditable, with no automatic looping or branching.

Owns:

- Iteration workflows
- Parent-child relationship tracking
- Audit trail completeness
- Regeneration control

Lead 10: Data Storage & Integrity Lead

This lead owns all persistence mechanisms within CAF. Their responsibility is to ensure that all entities—questions, contexts, instructions, versions, variants, feedback, and iteration links—are stored correctly and consistently.

They enforce referential integrity, prevent orphaned records, and ensure data reliability across JSON or SQLite storage. This lead also oversees data structure documentation.

Owns:

- Storage schema design
- JSON / SQLite persistence
- Referential integrity enforcement
- Data consistency validation

Lead 11: Internal Review & Acceptance Lead

This lead is responsible for final internal validation against the SRS acceptance criteria. They ensure that all functional and non-functional requirements are met and that no prohibited behaviors exist anywhere in the system.

They coordinate internal reviews, verify audit readiness, and ensure that documentation is complete and accurate before handoff.

Owns:

- Acceptance criteria validation
- Internal review coordination
- Documentation readiness
- Final system sign-off